Here is the complete technology stack used to build **Sopaan**, detailing each layer, component, and its specific role in the architecture.

SOPAAN: Complete Technology Stack & Role Matrix

Layer / Domain	Technology / Tool	Role in the Architecture
Frontend & UI (Client)	**Vite + Preact + TypeScript**	Powers the student PWA and facilitator interface. Chosen for its lightweight bundle size (< 2 MB gzipped), ensuring fast interactive load times on low-end 2 GB Android devices.
Service Worker & Caching	**Workbox**	Handles offline-first capabilities by precaching the app shell and managing runtime caching for signed packs and audio clips.
Local Client Storage	**Dexie / IndexedDB**	Acts as the local system of record for each session on the device. Manages local database schemas (profiles, packs, wallet credentials, outbox logs) and enforces transaction integrity via the write-ahead log (WAL) property.
Client Rules & Logic Engine	**json-logic-js (with a custom 3-valued wrapper)**	Evaluates scheme eligibility locally on the device without a network. Processes criteria trees into TRUE, FALSE, or NEED_INFO states.
Cryptography & Signatures	**@noble/ed25519 (Client) / JDK Ed25519 & PyNaCl (Server)**	Provides uniform Ed25519 digital signatures and SHA-256 hashes across all runtimes (JavaScript, Java,

		Python). Secures rule packs, credentials, and event envelopes so trust lives in the signature rather than the channel.
Voice & Speech Pipeline	**Vosk WASM + Web APIs (speechSynthesis, AudioWorklet)**	Provides offline-first speech-to-text (STT) processing using closed-vocabulary slot grammars for local languages (Hindi/Marathi) alongside pre-recorded audio clips and device TTS for read-backs.
Backend Core	**Spring Boot 3 (Java 17+)**	Implements the modular monolith core divided into clear bounded packages: rules, passport, sync, cohort, credits, and audit. Operates under two profiles: cloud (server) and hub (facilitator laptop).
AI & Tooling Service	**FastAPI + Pydantic**	Operates as an optional, isolated sidecar service handling heavy or uncertain tasks: automated rule extraction from documents (/rules:extract), profile slot extraction, grounded explanations, and fairness/CI test harnesses.
Database & Persistence	**Neon Postgres (Cloud) / H2 or Local Postgres (Hub)**	Serves as the system of record on the cloud (using Neon’s scale-to-zero free tier) and a local file database on the facilitator hub, maintaining complete schema parity.

Database Migrations	**Flyway**	Manages versioned database schema migrations consistently across cloud and hub environments.
CI/CD & Delivery	**Docker Compose & GitHub Actions**	Automates container packaging for the facilitator hub, executes automated testing/golden-test parity checks, and handles build-time pack signing and deployment.
Hosting & Infrastructure	**Static Host (GitHub/Cloudflare Pages) + Render (Free Tiers)**	Hosts the static PWA bundle and signed packs on a static CDN, while running the Spring Boot and FastAPI backends on Render's free web service tier.
Testing Frameworks	**JUnit, fast-check, Playwright, k6, ZAP Baseline**	Powers unit tests, property-based tests, offline end-to-end browser tests, API load testing under sync bursts, and security/authorization baseline checks.